Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине «Проектирование и реализация баз данных»

Тема: Запросы на выборку и модификацию данных. Представления. Работа с индексами

Выполнила Мкртчян К.Г.; К3241.

Проверили Говорова М.М., Белов А.О.

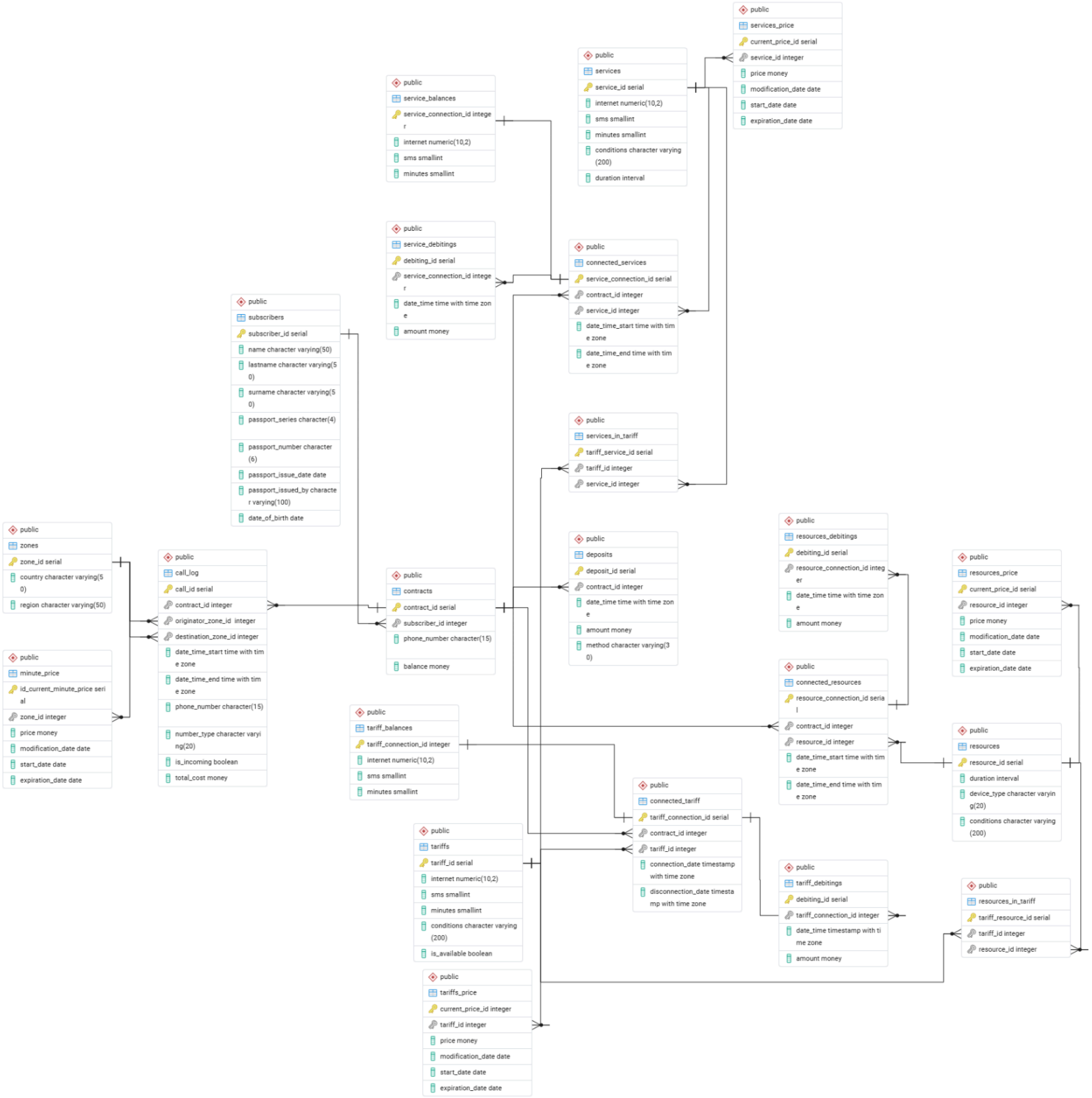
Дата 21.04.2026

Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

- Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
- Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
- Изучить графическое представление запросов и посмотреть историю запросов.
- Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

ERD- диаграмма



Задание 2. Создать запросы:

1. Вывести суммарное время переговоров каждого абонента за заданный период.

```
select subscriber_id, coalesce(sum(date_time_end -
date_time_start), '00:00:00') as sum_time from contracts

left join call_log on contracts.contract_id =
call_log.contract_id

and date_time_start between '2026-03-05 00:00:00+03' and
'2026-04-05 00:00:00+03'

group by subscriber_id

order by sum_time
```

	subscriber_id integer	sum_time interval
74	1896	00:00:00
75	1820	00:00:00
76	1841	00:00:00
77	1947	00:01:00
78	1886	00:01:00
79	1889	00:02:00

*`coalesce` - выбирает первое выражение не равное null

2. Найти среднюю продолжительность разговора абонента АТС.

```
select subscriber_id,  avg(date_time_end -
date_time_start) as avg_time from contracts

left join call_log on contracts.contract_id =
call_log.contract_id and number_type = 'городской'

group by subscriber_id

having avg(date_time_end - date_time_start) is not null

order by avg_time
```

	subscriber_id integer	avg_time interval
1	1912	00:01:00
2	1929	00:01:00
3	1877	00:02:00
4	1807	00:02:00
5	1919	00:04:30

3. Вывести количество междугородных переговоров каждого абонента.

```
select subscriber_id, count(originator_zone_id) from
contracts

left join call_log on contracts.contract_id =
call_log.contract_id

and originator_zone_id <> destination_zone_id

group by subscriber_id

order by subscriber_id
```

	subscriber_id integer	count bigint
75	1875	4
76	1876	0
77	1877	2
78	1878	1
79	1879	2
80	1880	2
81	1881	3

4. Вывести список абонентов, продолжительность разговоров которых за прошедшую неделю была максимальной. (взяла данные за 31 день, для недели будет interval ‘7 days’)

```
with subscriber_totals as (

    select subscriber_id, sum(date_time_end -
date_time_start) as total_time from contracts

    left join call_log on call_log.contract_id =
contracts.contract_id and date_time_start >= current_date
- interval '31 days'
```

```
    group by subscriber_id
)
select subscriber_id, total_time
from subscriber_totals
where total_time >= (select max(total_time) from
subscriber_totals);
```

	subscriber_id integer	total_time interval
1	1861	01:40:00

5. Сколько звонков было сделано в каждый из городов России.

```
select region, count(call_id) from zones
left join call_log on call_log.destination_zone_id =
zones.zone_id
where country = 'Россия'
group by region
```

	region character varying (50)	count bigint
1	Новосибирская облас...	11
2	Татарстан	12
3	Краснодарский край	13
4	Иркутская область	12
5	Красноярский край	11
6	Свердловская область	14
7	Москва	15
8	Ростовская область	12
9	Владивосток	14
10	Санкт-Петербург	11

6. Вывести список абонентов, звонивших только в ночное время.

```
select distinct contracts.subscriber_id, subscribers.name
from contracts
```

```
left join subscribers on subscribers.subscriber_id =
contracts.subscriber_id

where contracts.subscriber_id not in (select distinct
subscriber_id from contracts

left join call_log on contracts.contract_id =
call_log.contract_id

where date_time_start::time >= '06:00:00') and
contracts.contract_id in (select distinct
call_log.contract_id from call_log)
```

	subscriber_id integer	name character varying (50)
1	1939	Владислав
2	1801	Анатолий
3	1869	Моисей
4	1823	Гурий
5	1858	Ярополк
6	1931	Викентий
7	1836	Федор
8	1896	Филимон
9	1917	Ия
10	1924	Иосиф

7. Вывести список абонентов, время разговоров которых превышает среднее для этой же зоны.

```
  select contracts.subscriber_id, zones.zone_id,
max(call_log.date_time_end - call_log.date_time_start),
count(*) from contracts

  join call_log on call_log.contract_id =
contracts.contract_id
  join zones on zones.zone_id = call_log.originator_zone_id
  where  call_log.date_time_end - call_log.date_time_start
> (select avg(date_time_end - date_time_start) from
call_log
  where call_log.originator_zone_id = zones.zone_id)

group by contracts.subscriber_id, zones.zone_id
```

	subscriber_id integer	zone_id integer	max interval	count bigint
1	1811	225	00:53:00	1
2	1915	240	00:55:00	1
3	1861	214	00:51:00	1
4	1830	235	00:59:00	2
5	1944	223	00:50:00	1

Задание 3. Создать представление:

1. Содержащее сведения обо всех абонентах и их переговорах за прошедший месяц.

```
create or replace view view1 as

select contracts.subscriber_id, contracts.contract_id,
date_time_start, date_time_end, call_log.phone_number,
is_incoming, number_type, total_cost, originator_zone_id,
destination_zone_id from contracts

left join call_log on contracts.contract_id =
call_log.contract_id

and date_time_start >= date_trunc('month', current_date)
- interval '1 month' and date_time_start <
date_trunc('month', current_date)
```

	subscriber_id integer	contract_id integer	date_time_start timestamp with time zone	date_time_end timestamp with time zone	phone_number character (15)	is_incoming boolean	number_type character varying (20)	total_cost money	originator_zone_id integer	destination_zone_id integer
1	1930	1930	2026-03-03 02:53:09+03	2026-03-03 03:00:09+03	+7-970-498-57...	true	мобильный	171,00 ?	234	216
2	1852	1852	2026-03-20 11:13:04+03	2026-03-20 11:58:04+03	+7-989-287-24...	false	городской	10,00 ?	235	230
3	1938	1938	2026-03-11 21:59:13+03	2026-03-11 22:23:13+03	+7-922-494-22...	true	городской	5,00 ?	224	232
4	1942	1942	2026-03-29 21:15:08+03	2026-03-29 21:57:08+03	+7-923-639-61...	true	городской	320,00 ?	227	227
5	1915	1915	2026-03-20 08:22:17+03	2026-03-20 09:17:17+03	+7-926-550-82...	false	мобильный	253,00 ?	240	222
6	1861	1861	2026-03-25 06:08:26+03	2026-03-25 06:39:26+03	+7-973-162-73...	true	мобильный	320,00 ?	215	222

2. Найти самую популярную зону звонков за истекший год.

```
create or replace view view2 as

select zone_id, country, region, count(call_id) from
call_log

join zones on zones.zone_id = call_log.originator_zone_id
```

```
and date_time_start >= date_trunc('year', current_date) -
interval '1 year' and date_time_start <
date_trunc('year', current_date)

group by zone_id, country, region

having count(call_id) >= all (

    select count(call_id)

    from call_log

    where date_time_start >= date_trunc('year',
current_date) - interval '1 year'

      and date_time_start < date_trunc('year',
current_date)

    group by originator_zone_id

)

order by zone_id
```

	zone_id integer	country character varying (50)	region character varying (50)	count bigint
1	211	Россия	Москва	2
2	234	Испания	Мадрид	2

Задание 4. Запросы на модификацию данных с использованием подзапросов:

1) Добавление новой записи в журнал звонков

	subscriber_id integer	contract_id integer	originator_zone_id integer	region character varying (50)	destination_zone_id integer	date_time_start timestamp with time zone	date_time_end timestamp with time zone	phone_number character (15)	number_type character varying (20)	is_incoming boolean	total_cost money
1	1811	1811	211	Москва	237	2026-03-04 20:59:46+03	2026-03-04 21:46:46+03	+7-931-952-21...	мобильный	true	499,00 ?
2	1827	1827	211	Москва	225	2026-03-08 01:25:22+03	2026-03-08 01:57:22+03	+7-967-872-53...	городской	false	328,00 ?
3	1827	1827	211	Москва	225	2026-01-25 10:46:21+03	2026-01-25 10:48:21+03	+7-991-924-41...	городской	false	376,00 ?

Запрос:

```
insert into call_log (contract_id, originator_zone_id,
destination_zone_id, date_time_start, date_time_end,
phone_number, number_type, is_incoming, total_cost)

select

    contracts.contract_id,
```

```
    (select zone_id from zones where region = 'Москва'
limit 1),

    (select zone_id from zones where region =
'Санкт-Петербург' limit 1),

    '2025-03-25 10:00:00',

    '2025-03-25 10:15:00',

    contracts.phone_number,

    'городской',

    false,

    50.00

from contracts

where contracts.subscriber_id = 1811;
```

	subscriber_id integer	contract_id integer	originator_zone_id integer	region character varying (50)	destination_zone_id integer	date_time_start timestamp with time zone	date_time_end timestamp with time zone	phone_number character (15)	number_type character varying (20)	is_incoming boolean	total_cost money
1	1811	1811	211	Москва	237	2026-03-04 20:59:46+03	2026-03-04 21:46:46+03	+7-931-952-21...	мобильный	true	499,00 ?
2	1811	1811	211	Москва	212	2025-03-25 10:00:00+03	2025-03-25 10:15:00+03	+7-943-673-38...	городской	false	50,00 ?
3	1827	1827	211	Москва	225	2026-03-08 01:25:22+03	2026-03-08 01:57:22+03	+7-967-872-53...	городской	false	328,00 ?

Запрос, который был сделан при выборке:

```
select subscriber_id, call_log.contract_id,
originator_zone_id, region, destination_zone_id,
date_time_start, date_time_end, call_log.phone_number,
number_type, is_incoming, total_cost from call_log

left join zones on zones.zone_id =
call_log.originator_zone_id

left join contracts on contracts.contract_id =
call_log.contract_id

where region = 'Москва'

order by subscriber_id
```

2) Обновление интернета (для переноса остатков интернета на следующий месяц)

	contract_id integer	balance money	tariff_id integer	internet_in_tariff numeric (10,2)	internet_balance numeric (10,2)
8	1808	0,00 ?	634	27.25	12.00
9	1809	0,00 ?	614	29.28	12.00
10	1810	0,00 ?	642	28.00	12.00
11	1811	6 759,00 ?	617	12.81	12.00
12	1812	1 085,00 ?	608	56.49	12.00

```
update tariff_balances
set internet = tariff_balances.internet + coalesce(
(
select tariffs.internet from contracts
left join connected_tariff on contracts.contract_id =
connected_tariff.contract_id
left join tariffs on tariffs.tariff_id =
connected_tariff.tariff_id
where connected_tariff.tariff_connection_id =
tariff_balances.tariff_connection_id and
contracts.balance > 0::money
), 0)
```

	contract_id integer	balance money	tariff_id integer	internet_in_tariff numeric (10,2)	internet_balance numeric (10,2)
8	1808	0,00 ?	634	27.25	12.00
9	1809	0,00 ?	614	29.28	12.00
10	1810	0,00 ?	642	28.00	12.00
11	1811	6 759,00 ?	617	12.81	24.81
12	1812	1 085,00 ?	608	56.49	68.49

Запрос на выборку:

```
select contracts.contract_id, contracts.balance,
tariffs.tariff_id, tariffs.internet as
internet_in_tariff, tariff_balances.internet as
internet_balance from contracts

left join connected_tariff on contracts.contract_id =
connected_tariff.contract_id

left join tariff_balances on
connected_tariff.tariff_connection_id =
tariff_balances.tariff_connection_id
```

```
left join tariffs on tariffs.tariff_id =
connected_tariff.tariff_id
```

3) Удаление цен неиспользуемого ресурса

	service_id integer	price money	connections_count bigint
3	453	100,00 ?	5
4	454	100,00 ?	0
5	455	100,00 ?	3
6	456	100,00 ?	9

```
delete from services_price

where services_price.service_id in

(select service_id from (select services.service_id,
price, count(connected_services.contract_id) as
connections_count from services

left join connected_services on
connected_services.service_id = services.service_id

left join services_price on services_price.service_id =
services.service_id

group by services.service_id, price

having count(connected_services.contract_id) = 0))
```

	service_id integer	price money	connections_count bigint
3	453	100,00 ?	5
4	454	[null]	0
5	455	100,00 ?	3
6	456	100,00 ?	9

Запрос для выборки данных:

```
select services.service_id, price,
count(connected_services.contract_id) as
connections_count from services
```

```
left join connected_services on
connected_services.service_id = services.service_id

left join services_price on services_price.service_id =
services.service_id

group by services.service_id, price
```

Задание 5: Индексы

Запрос:

```
explain analyze

select * from call_log where contract_id = 100;
```

	QUERY PLAN text
1	Seq Scan on call_log (cost=0.00..11.00 rows=1 width=76) (actual time=0.160..0.161 rows=0.00 loop...
2	Filter: (contract_id = 100)
3	Rows Removed by Filter: 400
4	Buffers: shared hit=6
5	Planning:
6	Buffers: shared hit=48
7	Planning Time: 2.237 ms
8	Execution Time: 0.216 ms

Простой индекс:

```
create index idx_call_log_contract_id on
call_log(contract_id);
```

Результат: время выполнения запроса уменьшилось в 3.5 раза

	QUERY PLAN text
1	Index Scan using idx_call_log_contract_id on call_log (cost=0.15..8.17 rows=1 width=76) (actual time=0.044..0.044 rows=0.00 loo...
2	Index Cond: (contract_id = 100)
3	Index Searches: 1
4	Buffers: shared read=1
5	Planning:
6	Buffers: shared hit=62 read=1
7	Planning Time: 1.999 ms
8	Execution Time: 0.078 ms

Удалили индекс:

```
drop index idx_call_log_contract_id;
```

Запрос:

```
explain analyze
select contract_id, count(*)
from call_log
where date_time_start >= '2025-01-01'
group by contract_id;
```

	QUERY PLAN text
1	HashAggregate (cost=13.00..14.41 rows=141 width=12) (actual time=0.249..0.271 rows=140.00 loops=1)
2	Group Key: contract_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=6
5	-> Seq Scan on call_log (cost=0.00..11.00 rows=400 width=4) (actual time=0.044..0.108 rows=400.00 loo...
6	Filter: (date_time_start >= '2025-01-01 00:00:00+03'::timestamp with time zone)
7	Buffers: shared hit=6
8	Planning:
9	Buffers: shared hit=27 dirtied=2
10	Planning Time: 1.627 ms
11	Execution Time: 0.325 ms

Составной индекс:

```
create index idx_call_log_contract_date on
call_log(contract_id, date_time_start);
```

Результат: время выполнения запроса уменьшилось в 2 раза

	QUERY PLAN text
1	HashAggregate (cost=13.00..14.41 rows=141 width=12) (actual time=0.162..0.179 rows=140.00 loops=1)
2	Group Key: contract_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=6
5	-> Seq Scan on call_log (cost=0.00..11.00 rows=400 width=4) (actual time=0.019..0.083 rows=400.00 loo...
6	Filter: (date_time_start >= '2025-01-01 00:00:00+03'::timestamp with time zone)
7	Buffers: shared hit=6
8	Planning:
9	Buffers: shared hit=39 read=1
10	Planning Time: 1.862 ms
11	Execution Time: 0.229 ms

Удалили индекс:

```
drop index idx_call_log_contract_date;
```